

Project: Real Time Computer Vision Pipeline

07-projects/realtime-cv-pipeline

Overview

A real time computer vision pipeline handles image or video related inference under tight latency constraints. It coordinates ingestion, preprocessing, model execution, response shaping, and operational monitoring.

This kind of system is harder than a notebook demo because latency, payload size, concurrency, and failure handling all become first class engineering concerns.

Core Concepts

- Core components usually include an API layer, image transport or storage, preprocessing logic, a model runtime with GPU or optimized CPU support, and a result sink for downstream systems.
- The design must account for payload size, serialization overhead, and whether inference happens synchronously on request or asynchronously after upload.
- Observability needs to distinguish infrastructure failures from model failures, such as corrupt inputs, low confidence outputs, or drift in camera conditions.

How It Works

- A client uploads an image or frame reference, the system validates and preprocesses it, and the model runtime performs detection, classification, or segmentation.
- The service returns structured predictions quickly or stores results for later retrieval if the payload and model latency make asynchronous processing more appropriate.
- Logs, request tracing, and sampled payload metadata support debugging when quality drops or latency spikes under production traffic.

AI and ML Relevance

- This project demonstrates the gap between model training success and production reality. Input normalization, batching strategy, and hardware choice often matter as much as the model architecture.
- It is also a useful pattern for understanding how AWS services combine around GPU inference, object storage, APIs, and edge facing reliability concerns.

Operational Notes

- Test with realistic image sizes and traffic bursts. Benchmarks based on tiny local samples routinely mislead teams.
- Protect the expensive inference layer with rate limits, queues, or admission control. One malformed client can otherwise create a cost incident.
- Version preprocessing code with the model. A mismatch between training time and serving time transforms silently damages quality.

What To Remember

- Real time CV is a systems problem, not just a model problem.
- Latency budget is shared across transport, preprocessing, inference, and response handling.
- The serving path must preserve the same assumptions the model saw during training, or quality will collapse in production.